PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 1

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Merge Sort (Order Timestamps)**	**Batch:**	A1

1. Problem Statement

Design and implement a sorting algorithm using **Merge Sort** to efficiently arrange customer orders based on their timestamps. The solution should handle a large dataset (up to 1 million orders) with minimal computational overhead. Additionally, analyze the time complexity and compare it with traditional sorting techniques.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory

Merge Sort is a widely used sorting algorithm known for its guaranteed efficiency and stability. It is based on the **Divide-and-Conquer** algorithmic paradigm. It works by recursively dividing the input array into two equal halves, recursively sorting both halves, and finally merging the two sorted subarrays into a single consolidated sorted array.

Working Principles of Merge Sort:

- **Divide:** Recursively partition the array into two halves at the midpoint until each sub-array contains a single element (a single element is trivially sorted).
- **Conquer:** Recursively sort the sub-arrays. The base condition is reached when sub-array size becomes 1.
- **Combine (Merge):** Merge the sorted sub-arrays by comparing elements from both halves sequentially and placing the smaller element into an auxiliary array until all elements are merged.

Complexity Analysis:

- **Divide step:** Dividing an array of size *n* takes O(1) time. The depth of the recursion tree is $\lfloor \log_2(n) \rfloor + 1$.
- **Merge step:** At each level of the recursion tree, all *n* elements are compared and copied, which takes O(n) time.
- **Total Time Complexity:** Total Work = (Tree Height) × (Work per Level) = $\log_2(n) \times n$ = **O(n log n)**.

Case	Time Complexity	Rationale
Best Case	O(n log n)	Even if elements are already ordered, the array is still recursively divided and merged.
Average Case	O(n log n)	Typical random distribution requires standard partitioning and linear merging at each level.
Worst Case	O(n log n)	Strictly bounded by O(n log n) regardless of the initial arrangement of keys.
Space Complexity	O(n)	Requires an auxiliary array of size *n* during the merge procedure.

5. Merge Sort Algorithm & Pseudo Code

```
procedure mergesort(array A, left, right):
    if left < right then
        mid ← left + (right - left) / 2
        mergesort(A, left, mid)
        mergesort(A, mid + 1, right)
        merge(A, left, mid, right)
end procedure

procedure merge(array A, left, mid, right):
    Create temporary arrays L and R
    Copy data from A[left..mid] to L and A[mid+1..right] to R
    i ← 0, j ← 0, k ← left
    while i < size(L) and j < size(R) do
        if L[i].timestamp ≤ R[j].timestamp then A[k] ← L[i]; i ← i + 1
        else A[k] ← R[j]; j ← j + 1
        k ← k + 1
    Copy any remaining elements of L and R into A
end procedure
```

6. Process Flow Diagram



7. Step-by-Step Example Walkthrough

Given Customer Order Timestamps: [12:30, 09:15, 14:45, 10:00]
Step 1 (Divide): Split into [12:30, 09:15] and [14:45, 10:00] → Further split into [12:30], [09:15], [14:45], [10:00].
Step 2 (Merge & Sort):
• Merge [12:30] & [09:15] → [09:15, 12:30]
• Merge [14:45] & [10:00] → [10:00, 14:45]
• Final Merge: Compare [09:15, 12:30] and [10:00, 14:45] → **[09:15, 10:00, 12:30, 14:45]**.

8. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 1: Sorting E-Commerce Orders by Timestamp using Merge Sort
 */

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

struct Order {
    int orderId;
    int customerId;
    string timestamp;
    double amount;
};

// Merge two sorted parts
void merge(Order orders[], int left, int mid, int right) {

    int n1 = mid - left + 1;
    int n2 = right - mid;

    Order* L = new Order[n1];
    Order* R = new Order[n2];

    // Copy data
    for (int i = 0; i < n1; i++)
        L[i] = orders[left + i];

    for (int i = 0; i < n2; i++)
        R[i] = orders[mid + 1 + i];

    int i = 0;
    int j = 0;
    int k = left;

    // Compare timestamps
    while (i < n1 && j < n2) {

        if (L[i].timestamp <= R[j].timestamp) {
            orders[k] = L[i];
            i++;
        }
        else {
            orders[k] = R[j];
            j++;
        }

        k++;
    }

    // Copy remaining elements
    while (i < n1) {
        orders[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        orders[k] = R[j];
        j++;
```

```
        k++;
    }

    delete[] L;
    delete[] R;
}

// Merge Sort
void mergeSort(Order orders[], int left, int right) {

    if (left < right) {

        int mid = left + (right - left) / 2;

        mergeSort(orders, left, mid);
        mergeSort(orders, mid + 1, right);

        merge(orders, left, mid, right);
    }
}

int main() {

    const int MAX_ORDERS = 10000;

    Order* orders = new Order[MAX_ORDERS];

    ifstream file("orders_sample.csv");

    if (!file) {
        cout << "Error: Could not open orders_sample.csv" << endl;
        return 1;
    }

    string line;

    // Skip header
    getline(file, line);

    int count = 0;

    // Read CSV file
    while (getline(file, line) && count < MAX_ORDERS) {

        stringstream ss(line);
        string value;

        // order_id
        getline(ss, value, ',');
        orders[count].orderId = stoi(value);

        // customer_id
        getline(ss, value, ',');
        orders[count].customerId = stoi(value);

        // order_timestamp
        getline(ss, value, ',');
        orders[count].timestamp = value;

        // amount
        getline(ss, value, ',');
        orders[count].amount = stod(value);

        count++;
    }

    file.close();
```

```
    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------\n";
    cout << "Total orders read: " << count << endl;

    // Sort using Merge Sort
    mergeSort(orders, 0, count - 1);

    cout << "\nOrders sorted by timestamp:\n\n";

    // Display first 20 orders
    int displayCount = min(count, 20);

    for (int i = 0; i < displayCount; i++) {

        cout << "Order ID: " << orders[i].orderId
             << " | Customer ID: " << orders[i].customerId
             << " | Timestamp: " << orders[i].timestamp
             << " | Amount: " << orders[i].amount
             << endl;
    }

    delete[] orders;

    return 0;
}
```

9. Sample Execution Output

```
======================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 1 - E-Commerce Orders Sorting using Merge Sort
======================================================

Total orders read: 10000

Orders sorted by timestamp:

Order ID: 4009 | Customer ID: 3507 | Timestamp: 2024-01-01T00:39:56 | Amount: 1209.56
Order ID: 6866 | Customer ID: 1295 | Timestamp: 2024-01-01T01:08:04 | Amount: 1876.5
Order ID: 8613 | Customer ID: 5438 | Timestamp: 2024-01-01T01:35:32 | Amount: 989.3
Order ID: 1447 | Customer ID: 2489 | Timestamp: 2024-01-01T03:09:49 | Amount: 1584.61
Order ID: 1887 | Customer ID: 1567 | Timestamp: 2024-01-01T03:38:48 | Amount: 1966
Order ID: 162 | Customer ID: 5978 | Timestamp: 2024-01-01T07:34:02 | Amount: 577.39
Order ID: 3513 | Customer ID: 1854 | Timestamp: 2024-01-01T09:59:25 | Amount: 1322.56
Order ID: 9735 | Customer ID: 8598 | Timestamp: 2024-01-01T12:35:49 | Amount: 586.41
Order ID: 1043 | Customer ID: 8009 | Timestamp: 2024-01-01T12:36:00 | Amount: 408.62
Order ID: 1848 | Customer ID: 9289 | Timestamp: 2024-01-01T13:25:06 | Amount: 761.19
Order ID: 9105 | Customer ID: 5926 | Timestamp: 2024-01-01T14:47:39 | Amount: 940.3
Order ID: 4486 | Customer ID: 3416 | Timestamp: 2024-01-01T15:43:06 | Amount: 404.81
Order ID: 2155 | Customer ID: 6885 | Timestamp: 2024-01-01T15:43:13 | Amount: 223.85
Order ID: 9977 | Customer ID: 5863 | Timestamp: 2024-01-01T15:49:00 | Amount: 1903.1
Order ID: 2250 | Customer ID: 1619 | Timestamp: 2024-01-01T16:10:26 | Amount: 1330.89
Order ID: 2357 | Customer ID: 2158 | Timestamp: 2024-01-02T01:25:12 | Amount: 772.83
Order ID: 3553 | Customer ID: 6692 | Timestamp: 2024-01-02T02:05:49 | Amount: 1267.2
Order ID: 5894 | Customer ID: 8881 | Timestamp: 2024-01-02T03:13:33 | Amount: 390.87
Order ID: 1131 | Customer ID: 7164 | Timestamp: 2024-01-02T03:15:15 | Amount: 1698.77
Order ID: 8489 | Customer ID: 4272 | Timestamp: 2024-01-02T03:27:11 | Amount: 1879.27
```

10. Conclusion

Merge Sort is a highly scalable, stable, and predictable sorting algorithm that organizes customer order records based on timestamps with guaranteed **O(n log n)** worst-case time complexity. Compared to $O(n^2)$ quadratic algorithms such as Bubble Sort or Insertion Sort, Merge Sort easily processes massive transaction logs (up to 1,000,000 orders) without CPU degradation, making it suitable for modern e-commerce distributed backends.

GitHub Repository Link

> **Repository Link:** https://github.com/siddhesh160606/DAA_lab